

Prueba Técnica de Desarrollo Full-Stack con IA

Título del Proyecto: Analizador Inteligente de Contenido de Imágenes

Objetivo Principal

El objetivo es crear una aplicación web **full-stack** que permita a los usuarios subir una imagen. La aplicación analizará la imagen utilizando un servicio de IA y mostrará al usuario una lista de etiquetas o "tags" que describen el contenido de la imagen.

Este proyecto evaluará la capacidad del candidato para:

- Diseñar y construir una API en el **backend**.
 - Crear una interfaz de usuario funcional en el **frontend**.
 - Integrar servicios de terceros (la API de IA).
 - Utilizar **Git** para el control de versiones de manera profesional.
 - Escribir código limpio, estructurado y documentado.
-

Requisitos Detallados

1. Backend

El candidato puede elegir la tecnología de su preferencia entre el stack de kushki (Ej: Node.js/Express, Python/Flask, go.).

- **API Endpoint:** Debe crear al menos un endpoint:
 - POST /api/analyze
 - **Función:** Este endpoint recibirá un archivo de imagen (multipart/form-data).
 - **Proceso:** Deberá tomar la imagen recibida, enviarla a un servicio externo de análisis de imágenes por IA y procesar la respuesta.
 - **Respuesta:** Debe devolver un objeto JSON al frontend con los resultados. Por ejemplo:

```
JSON
{
  "tags": [
    { "label": "Perro", "confidence": 0.98 },
    { "label": "Golden Retriever", "confidence": 0.95 },
    { "label": "Parque", "confidence": 0.91 },
    { "label": "Césped", "confidence": 0.88 }
  ]
}
```

- **Integración con IA:**
 - El candidato **no necesita entrenar un modelo**. Debe utilizar una API de un servicio de IA pre-entrenado.
 - **Opciones sugeridas (puede usar otra):**
 - **Google Cloud Vision API:** Ofrece una capa gratuita generosa para el etiquetado de imágenes.
 - **Imagga:** También tiene un plan gratuito para desarrolladores.
 - **OpenAI API (GPT-4o Vision):** Puede describir imágenes de forma muy detallada.
 - La clave de la API **no debe** estar hardcodeada en el código fuente; debe ser gestionada a través de variables de entorno.
-

2. Frontend

El candidato puede elegir el framework de su preferencia (Ej: React, Vue, Angular) o incluso usar JavaScript puro (Vanilla JS).

- **Interfaz de Usuario:** La UI debe contener, como mínimo:
 - Un formulario que permita al usuario **seleccionar y subir un archivo de imagen** desde su computadora.
 - Un botón para "Analizar".
 - Un **indicador de carga** (spinner, barra de progreso, etc.) que se muestre mientras el backend y la IA procesan la imagen.
 - Una sección para **mostrar los resultados**. Debe mostrar tanto la imagen que se subió como la lista de etiquetas (tags) devueltas por el backend.
 - **Comunicación:** El frontend debe comunicarse con el endpoint del backend (/api/analyze) para enviar la imagen y recibir los resultados de forma asíncrona.
-

3. Control de Versiones (Git)

La forma en que el candidato gestiona el proyecto es tan importante como el resultado final.

- **Repositorio:** El proyecto completo debe ser entregado a través de un enlace a un repositorio de **Git** (ej. en GitHub, GitLab).
 - **Historial de Commits:** Se evaluará la calidad del historial. Se esperan **commits atómicos** con mensajes claros y descriptivos que expliquen el "qué" y el "porqué" de cada cambio.
 - **Ramas (Branches):** Se valorará positivamente el uso de ramas para el desarrollo de funcionalidades (ej. feature/backend-api, feature/frontend-uploader).
-

Entregables

1. **URL del Repositorio de Git:** El repositorio debe ser público o se debe dar acceso al evaluador.
 2. **Un archivo README.md claro y conciso** en la raíz del proyecto que debe incluir:
 - Una breve descripción del proyecto.
 - La lista de tecnologías utilizadas (ej. Node.js v18, React v18).
 - **Instrucciones detalladas** para instalar las dependencias y ejecutar el proyecto localmente (tanto backend como frontend).
 - Instrucciones sobre cómo configurar las variables de entorno necesarias (como la API Key del servicio de IA).
-

Criterios de Evaluación

- **Funcionalidad Completa:** ¿La aplicación cumple con todos los requisitos básicos?
- **Calidad del Código (Backend y Frontend):** Estructura del proyecto, legibilidad, uso de buenas prácticas, modularidad.
- **Manejo de Errores:** ¿Cómo gestiona la aplicación los posibles errores (ej. subir un archivo que no es una imagen, fallo en la API de IA)?
- **Calidad del Repositorio de Git:** Claridad y profesionalismo en los mensajes de commit y la estructura del repositorio.
- **Calidad del README.md:** ¿Las instrucciones son claras y completas? ¿Un nuevo desarrollador podría ejecutar el proyecto sin problemas?
- **Experiencia de Usuario (UX):** ¿La aplicación es intuitiva? ¿El indicador de carga mejora la experiencia?

Puntos Extra (Opcional)

Estos puntos no son obligatorios, pero demuestran un nivel de habilidad superior:

- **Contenerización:** Proveer un archivo Dockerfile y docker-compose.yml para ejecutar la aplicación fácilmente.
- **Pruebas (Testing):** Añadir pruebas unitarias o de integración para la API del backend.
- **Diseño y CSS:** Una interfaz de usuario pulida y responsiva (adaptable a móviles).
- **Seguridad Básica:** Implementar validaciones en el backend para el tipo y tamaño del archivo subido.